
INTRODUCING GAZEBO



Gazebo: 3D physics simulation tool that work with ROS.

- It creates a virtual world with real physics.
- Every sensor (LiDar, camera, IMU) produces realistic data.
- Every command produces realistic movement

Gazebo runs as separate process, connecting to ROS through a plugin

```
Real robot:                                Gazebo simulation:

Physical LiDAR                             Gazebo LiDAR plugin
  ↓ publishes                               ↓ publishes
  Topic: /scan                             = Topic: /scan

Topic: /cmd_vel                             = Topic: /cmd_vel
  ↓ moves robot                             ↓ moves virtual robot
Physical wheels                             Gazebo physics engine
```

INTRODUCING GAZEBO



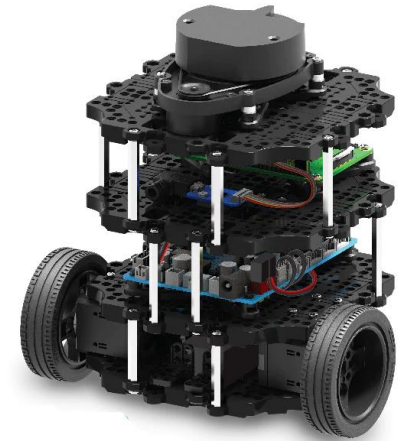
Install TurtleBot3 simulation packages

TurtleBot3 is a small differential-drive robot made by ROBOTIS.

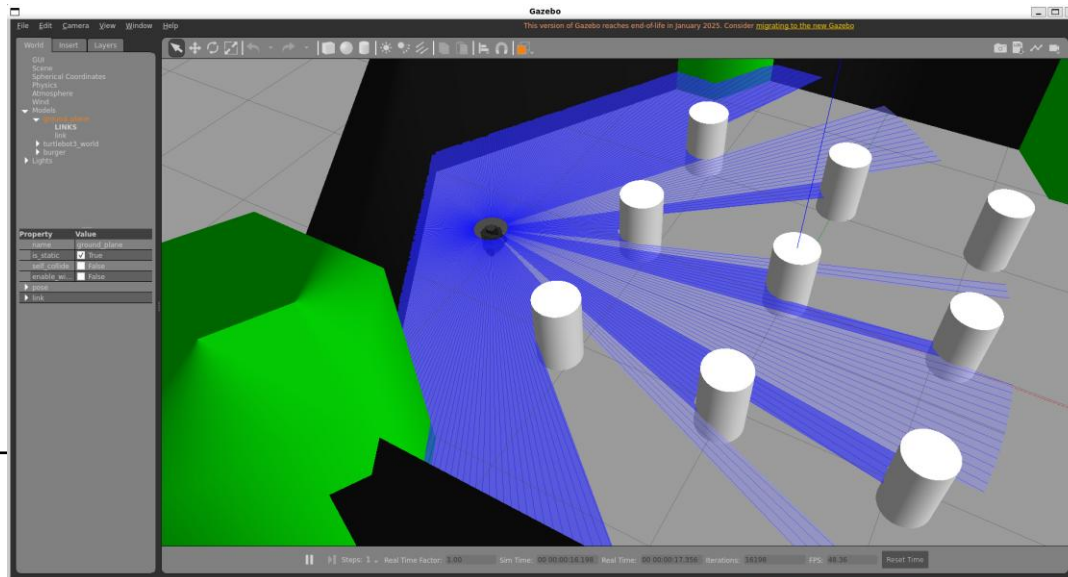
```
sudo apt update && sudo apt install -y ros-humble-turtlebot3*  
echo 'export TURTLEBOT3_MODEL=burger' >> ~/.bashrc && source ~/.bashrc
```

Test the model in Gazebo

```
ros2 launch turtlebot3_gazebo turtlebot3_world.launch.py
```



TurtleBot3 Burger



CREATE GAZEBO WORLD



Gazebo worlds are written in SDF (Simulation Description Format): XML file that describe everything in the scene (ground, lights, walls, etc.)

Every model in SDF has:

- pose : position and rotation in the world
- link : the physical body
- static : true – object does not move

Create the Workshop Package:

```
cd ~/ros2_ws/src
ros2 pkg create workshop gazebo --build-type ament cmake
mkdir -p ~/ros2_ws/src/workshop_gazebo/worlds
mkdir -p ~/ros2_ws/src/workshop_gazebo/launch
```

CREATE GAZEBO WORLD



Create the Classroom World:

```
nano ~/ros2_ws/src/workshop_gazebo/worlds/classroom.world
```

Create Sun (light):

```
<?xml version='1.0'>
<sdf version='1.6'>
  <world name='classroom_world'>
    <light name="sun" type="directional">
      <cast_shadows>true</cast_shadows>
      <pose>0 0 10 0 0 0</pose>
      <diffuse>0.8 0.8 0.8 1</diffuse>
      <specular>0.2 0.2 0.2 1</specular>
      <direction>-0.5 0.1 -0.9</direction>
    </light>
```

Create Gravity:

```
<gravity>0 0 -9.8</gravity>
```

Create Ground Floor:

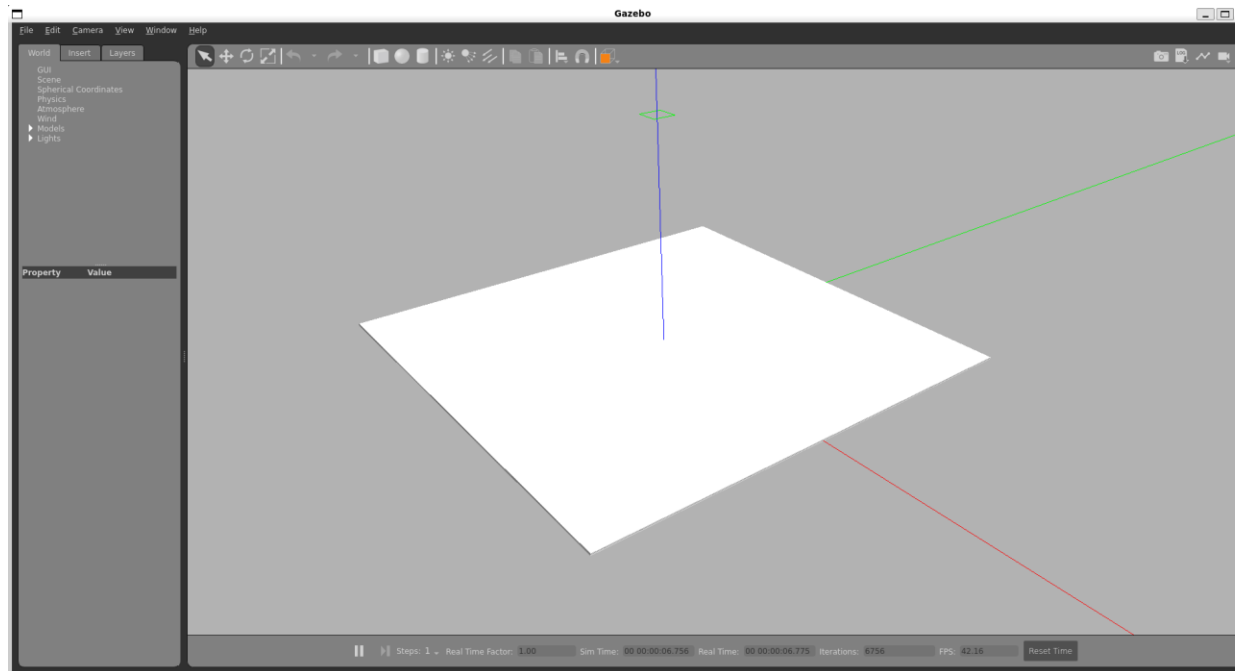
```
<model name="ground">
  <static>true</static>
  <pose>0 0 0 0 0 0</pose>
  <link name="ground_link">
    <collision name="ground_collision">
      <geometry>
        <box>
          <size>20 20 0.1</size>
        </box>
      </geometry>
    </collision>
    <visual name="ground_visual">
      <geometry>
        <box>
          <size>20 20 0.1</size>
        </box>
      </geometry>
    </visual>
  </link>
</model>
</world>
</sdf>
```

CREATE GAZEBO WORLD



Test the world

```
gazebo ~/ros2_ws/src/workshop_gazebo/worlds/classroom.world
```



CREATE GAZEBO WORLD



Create Four Walls:

```
<!-- Back wall -->
<model name="back_wall">
  <static>true</static> <!-- Wall is fixed in space -->
  <pose>0 -3 0.5 0 0 0</pose> <!-- Position at back side -->
  <link name="link">
    <collision name="collision"> <!-- Physics collision -->
      <geometry><box><size>6 0.2 1</size></box></geometry> <!-- Thin vertical wall -->
    </collision>
    <visual name="visual"> <!-- Visible wall -->
      <geometry><box><size>6 0.2 1</size></box></geometry>
    </visual>
  </link>
</model>
```

```
<!-- Left wall -->
<model name="left_wall">
  <static>true</static>
  <pose>-3 0 0.5 0 0 0</pose> <!-- Left boundary -->
  <link name="link">
    <collision name="collision">
      <geometry><box><size>0.2 6 1</size></box></geometry>
    </collision>
    <visual name="visual">
      <geometry><box><size>0.2 6 1</size></box></geometry>
    </visual>
  </link>
</model>
```

```
<!-- Front wall -->
<model name="front_wall">
  <static>true</static>
  <pose>0 3 0.5 0 0 0</pose> <!-- Front boundary -->
  <link name="link">
    <collision name="collision">
      <geometry><box><size>6 0.2 1</size></box></geometry>
    </collision>
    <visual name="visual">
      <geometry><box><size>6 0.2 1</size></box></geometry>
    </visual>
  </link>
</model>
```

```
<!-- Right wall -->
<model name="right_wall">
  <static>true</static>
  <pose>3 0 0.5 0 0 0</pose> <!-- Right boundary -->
  <link name="link">
    <collision name="collision">
      <geometry><box><size>0.2 6 1</size></box></geometry>
    </collision>
    <visual name="visual">
      <geometry><box><size>0.2 6 1</size></box></geometry>
    </visual>
  </link>
</model>
```

CREATE GAZEBO WORLD

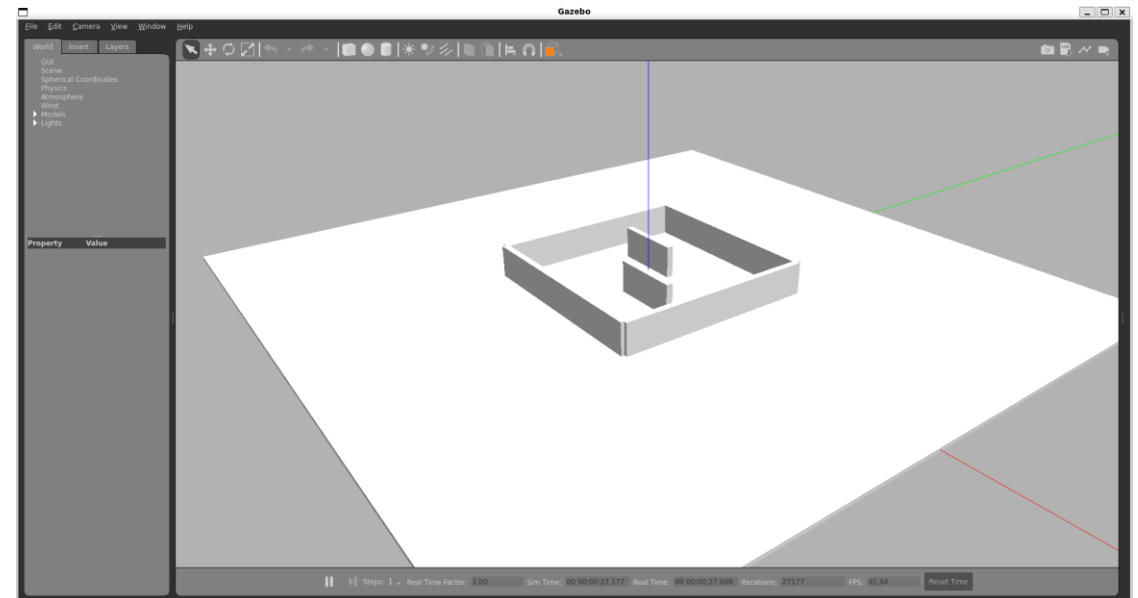


Adding the Obstacles:

```
<!-- Inner obstacle 1 -->
<model name="inner_wall_1">
  <static>true</static>
  <pose>-1 0.8 0.5 0 0 0</pose> <!-- Internal obstacle position -->
  <link name="link">
    <collision name="collision">
      <geometry><box><size>2.0 0.2 1</size></box></geometry> <!-- Thin barrier -->
    </collision>
    <visual name="visual">
      <geometry><box><size>2.0 0.2 1</size></box></geometry>
    </visual>
  </link>
</model>

<!-- Inner obstacle 2 -->
<model name="inner_wall_2">
  <static>true</static>
  <pose>1 -0.8 0.5 0 0 0</pose> <!-- Second internal obstacle -->
  <link name="link">
    <collision name="collision">
      <geometry><box><size>2.0 0.2 1</size></box></geometry>
    </collision>
    <visual name="visual">
      <geometry><box><size>2.0 0.2 1</size></box></geometry>
    </visual>
  </link>
</model>
```

Outcome:



CREATE GAZEBO WORLD



Create Launch File to Spawn Robot:

```
nano ~/ros2_ws/src/workshop_gazebo/launch/classroom_world.launch.py
```

```
import os

from launch import LaunchDescription
from launch.actions import ExecuteProcess, SetEnvironmentVariable, TimerAction
from launch_ros.actions import Node
from ament_index_python.packages import get_package_share_directory

def generate_launch_description():
    world_path = os.path.join(
        get_package_share_directory('workshop_gazebo'),
        'worlds',
        'classroom.world'
    )

    turtlebot3_gazebo_dir = get_package_share_directory('turtlebot3_gazebo')

    gazebo = ExecuteProcess(
        cmd=[
            'gazebo',
            '--verbose',
            '-s', 'libgazebo_ros_init.so',
            '-s', 'libgazebo_ros_factory.so',
            world_path
        ],
        output='screen'
    )
```

```
    spawn_turtlebot = Node(
        package='gazebo_ros',
        executable='spawn_entity.py',
        arguments=[
            '-entity', 'turtlebot3_burger',
            '-file', os.path.join(
                turtlebot3_gazebo_dir,
                'models',
                'turtlebot3_burger',
                'model.sdf'
            ),
            '-x', '-2.0',
            '-y', '-2.0',
            '-z', '0.01'
        ],
        output='screen'
    )

    return LaunchDescription([
        SetEnvironmentVariable(
            name='GAZEBO_MODEL_DATABASE_URI',
            value=''
        ),
        SetEnvironmentVariable(
            name='GAZEBO_MODEL_PATH',
            value=os.path.join(turtlebot3_gazebo_dir, 'models')
        ),
        gazebo,
        TimerAction(period=5.0, actions=[spawn_turtlebot])
    ])
```

CREATE GAZEBO WORLD



Register World Files in CMakeLists.txt: Tell Cmake to install the worlds/ and launch/ folders

```
nano ~/ros2_ws/src/workshop_gazebo/CMakeLists.txt
```

```
cmake_minimum_required(VERSION 3.8)
project(workshop_gazebo)

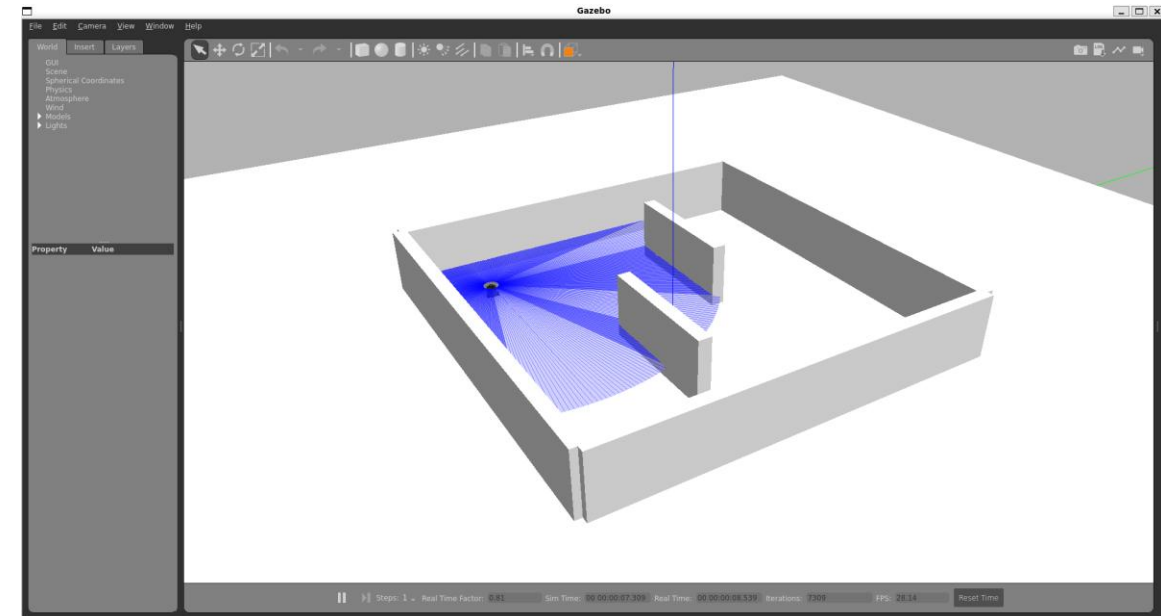
find_package(ament_cmake REQUIRED)

install(
  DIRECTORY worlds launch maps config
  DESTINATION share/${PROJECT_NAME}
)

ament_package()
```

Build and Launch:

```
cd ~/ros2_ws && colcon build && source install/setup.bash
ros2 launch workshop_gazebo classroom_world.launch.py
```



READING ROBOT SENSORS

Check what topics are being published by robots:

```
ros2 topic list
```

Expected output:

```
/clock  
/cmd_vel  
/imu  
/joint_states  
/odom  
/parameter_events  
/performance_metrics  
/rosout  
/scan  
/tf
```

robot movement

LiDAR

Create the Sensor Package:

```
cd ~/ros2 ws/src  
ros2 pkg create workshop_sensors --build-type ament_python \  
  --dependencies rclpy sensor_msgs nav_msgs
```

READING ROBOT SENSORS: LIDAR READER

nano ~/ros2_ws/src/workshop_sensors/workshop_sensors/lidar_reader.py

```
import math
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import LaserScan

class LidarReader(Node):
    def __init__(self):
        super().__init__('lidar_reader')
        self.sub = self.create_subscription(LaserScan, '/scan', self.scan_callback, 10)

    def scan_callback(self, msg):
        ranges = msg.ranges
        front = ranges[0:10] + ranges[-10:] # take 0 to 9 degree + 350 -359 degree
        left = ranges[80:100]
        right = ranges[260:280]
```

```
    def clean_min(values):
        valid = [v for v in values if not math.isinf(v) and not math.isnan(v)]
        return min(valid) if valid else float('inf')

    front_dist = clean_min(front)
    left_dist = clean_min(left)
    right_dist = clean_min(right)

    self.get_logger().info(
        f'Front: {front_dist:.2f} m | Left: {left_dist:.2f} m | Right: {right_dist:.2f} m'
    )

def main(args=None):
    rclpy.init(args=args)
    node = LidarReader()
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()
```

READING ROBOT SENSORS: ODOMETRY READER

nano ~/ros2_ws/src/workshop_sensors/workshop_sensors/odom_reader.py

```
import rclpy
from rclpy.node import Node
from nav_msgs.msg import Odometry
```

```
class OdomReader(Node):
    def __init__(self):
        super().__init__('odom_reader')
        self.sub = self.create_subscription(
            Odometry,
            '/odom',
            self.odom_callback,
            10
        )
```

```
    def odom_callback(self, msg):
        x = msg.pose.pose.position.x
        y = msg.pose.pose.position.y
        z = msg.pose.pose.position.z

        self.get_logger().info(
            f'Position: x={x:.2f}, y={y:.2f}, z={z:.2f}'
        )
```

```
def main(args=None):
    rclpy.init(args=args)
    node = OdomReader()
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()
```

READING ROBOT SENSORS: IMU READER

nano ~/ros2_ws/src/workshop_sensors/workshop_sensors/imu_reader.py

```
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import Imu

class ImuReader(Node):
    def __init__(self):
        super().__init__('imu_reader')
        self.sub = self.create_subscription(
            Imu,
            '/imu',
            self.imu_callback,
            10
        )
```

```
    def imu_callback(self, msg):
        wz = msg.angular_velocity.z
        ax = msg.linear_acceleration.x
        ay = msg.linear_acceleration.y

        self.get_logger().info(
            f'Angular z={wz:.4f} | Accel x={ax:.3f}, y={ay:.3f}'
        )

def main(args=None):
    rclpy.init(args=args)
    node = ImuReader()
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()
```

READING ROBOT SENSORS: BUILD AND RUN

```
cd ~/ros2_ws && colcon build && source install/setup.bash
```

Terminal 1:

```
ros2 launch workshop_gazebo classroom_world.launch.py
```

Terminal 2:

```
ros2 run workshop_sensors lidar_reader
```

Expected Output:

```
[lidar_reader]: Front: 2.85 m | Left: 2.91 m | Right: 2.98 m  
[lidar_reader]: Front: 2.85 m | Left: 2.91 m | Right: 2.98 m
```

CONTROLLING THE ROBOT

Create the Controller Package:

```
cd ~/ros2_ws/src

ros2 pkg create workshop_controller --build-type ament_python \
  --dependencies rclpy geometry_msgs sensor_msgs
```

Create Basic Motor Controller Node:

nano ~/ros2_ws/src/workshop_controller/workshop_controller/motor_controller.py

```
import rclpy
from rclpy.node import Node
from geometry_msgs.msg import Twist

class MotorController(Node):

    def __init__(self):
        super().__init__('motor_controller')
        self.pub = self.create_publisher(Twist, '/cmd_vel', 10)
        # Call control_loop every 0.1 seconds (10 Hz)
        self.create_timer(0.1, self.control_loop)
```

```
    def control_loop(self):
        msg = Twist()
        msg.linear.x = 0.15    # 0.15 m/s forward
        msg.angular.z = 0.0    # no rotation
        self.pub.publish(msg)

def main(args=None):
    rclpy.init(args=args)
    rclpy.spin(MotorController())
    rclpy.shutdown()
```

CONTROLLING THE ROBOT

Create LiDAR-Safe Controller Node:

nano ~/ros2_ws/src/workshop_controller/workshop_controller/obstacle_avoidance.py

```
import math
import rclpy
from rclpy.node import Node
from geometry_msgs.msg import Twist
from sensor_msgs.msg import LaserScan

class ObstacleAvoidance(Node):
    def __init__(self):
        super().__init__('obstacle_avoidance')

        self.front_dist = math.inf
        self.left_dist = math.inf
        self.right_dist = math.inf

        self.cmd_pub = self.create_publisher(Twist, '/cmd_vel', 10)

        self.scan_sub = self.create_subscription(
            LaserScan,
            '/scan',
            self.scan_callback,
            10
        )

        self.timer = self.create_timer(0.1, self.control_loop)
```

```
def clean_min(self, values):
    valid = [
        v for v in values
        if not math.isinf(v) and not math.isnan(v)
    ]
    return min(valid) if valid else math.inf

def scan_callback(self, msg):
    ranges = msg.ranges

    front = ranges[0:15] + ranges[-15:]
    left = ranges[70:110]
    right = ranges[250:290]

    self.front_dist = self.clean_min(front)
    self.left_dist = self.clean_min(left)
    self.right_dist = self.clean_min(right)
```

```
def control_loop(self):
    cmd = Twist()

    if self.front_dist < 0.45:
        cmd.linear.x = 0.0

        if self.left_dist > self.right_dist:
            cmd.angular.z = 0.6
            direction = 'turning left'
        else:
            cmd.angular.z = -0.6
            direction = 'turning right'

        self.get_logger().info(
            f'Obstacle ahead: {self.front_dist:.2f} m, {direction}'
        )

    else:
        cmd.linear.x = 0.18
        cmd.angular.z = 0.0

    self.cmd_pub.publish(cmd)

def main(args=None):
    rclpy.init(args=args)
    node = ObstacleAvoidance()
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()
```

CONTROLLING THE ROBOT

Create Wall Follower Controller Node using P Control:

nano ~/ros2_ws/src/workshop_controller/workshop_controller/wall_follower.py

```
import math
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import LaserScan
from geometry_msgs.msg import Twist

class WallFollower(Node):
    def __init__(self):
        super().__init__('wall_follower')

        self.left_distance = 0.0
        self.front_distance = 0.0

        self.target_distance = 0.25
        self.kp = 0.2

        self.cmd_pub = self.create_publisher(Twist, '/cmd_vel', 10)

        self.scan_sub = self.create_subscription(LaserScan, '/scan', self.scan_callback, 10)

        self.timer = self.create_timer(0.1, self.control_loop)

    def clean_min(self, values):
        valid = [v for v in values if not math.isinf(v) and not math.isnan(v)]
        return min(valid) if valid else 10.0
```

```
    def scan_callback(self, msg):
        ranges = msg.ranges

        self.front_distance = self.clean_min(
            ranges[0:10] + ranges[-10:]
        )

        self.left_distance = self.clean_min(
            ranges[80:100]
        )

    def control_loop(self):
        cmd = Twist()

        if self.front_distance < 0.3:
            cmd.linear.x = 0.0
            cmd.angular.z = -0.7
        else:
            error = self.target_distance - self.left_distance
            cmd.linear.x = 0.15
            cmd.angular.z = self.kp * error

        self.cmd_pub.publish(cmd)

def main(args=None):
    rclpy.init(args=args)
    node = WallFollower()
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()
```

A* PATH PLANNING

■ ■ How A* Works — The Core Idea

A* searches a grid map. Each cell is either **free (0)** or **occupied (100)**. The algorithm finds the shortest path.

It scores each cell with: $f(n) = g(n) + h(n)$

- $g(n)$ — actual cost to reach this cell from start
- $h(n)$ — estimated cost from this cell to goal (heuristic)
- $f(n)$ — total estimated path cost through this cell

Always expand the cell with the lowest $f(n)$. This guarantees the shortest path when it exists.

Pipeline:

```
Our classroom world (SDF walls)
    ↓
Map Publisher → /map topic (OccupancyGrid)
    ↓
A* Planner → /planned_path (Path of waypoints)
    ↓
Waypoint Follower → /cmd_vel (Robot moves)
    ↓
Robot reaches goal
```

A* PATH PLANNING

Create the Planner Package:

```
cd ~/ros2_ws/src

ros2 pkg create workshop_planner --build-type ament_python \
  --dependencies rclpy geometry_msgs nav_msgs sensor_msgs

ros2 pkg create workshop_mapping --build-type ament_python \
  --dependencies rclpy nav_msgs geometry_msgs
```

Create map_publisher:

nano ~/ros2_ws/src/workshop_mapping/workshop_mapping/known_map_publisher.py

```
import rclpy
from rclpy.node import Node
from nav_msgs.msg import OccupancyGrid
from geometry_msgs.msg import Pose

class KnownMapPublisher(Node):
    def __init__(self):
        super().__init__('known_map_publisher')

        self.map_pub = self.create_publisher(
            OccupancyGrid,
            '/map',
            10
        )

        self.resolution = 0.05
        self.width = 120
        self.height = 120
        self.origin_x = -3.0
        self.origin_y = -3.0

        self.timer = self.create_timer(1.0, self.publish_map)

    def world_to_grid(self, x, y):
        gx = int((x - self.origin_x) / self.resolution)
        gy = int((y - self.origin_y) / self.resolution)
        return gx, gy
```

```
def set_obstacle_rect(self, grid, center_x, center_y, size_x, size_y):
    min_x = center_x - size_x / 2.0
    max_x = center_x + size_x / 2.0
    min_y = center_y - size_y / 2.0
    max_y = center_y + size_y / 2.0

    gx_min, gy_min = self.world_to_grid(min_x, min_y)
    gx_max, gy_max = self.world_to_grid(max_x, max_y)

    gx_min = max(0, min(self.width - 1, gx_min))
    gx_max = max(0, min(self.width - 1, gx_max))
    gy_min = max(0, min(self.height - 1, gy_min))
    gy_max = max(0, min(self.height - 1, gy_max))

    for gy in range(gy_min, gy_max + 1):
        for gx in range(gx_min, gx_max + 1):
            index = gy * self.width + gx
            grid[index] = 100 #occupied (walls)
```

```
def publish_map(self):
    msg = OccupancyGrid()

    msg.header.frame_id = 'odom'
    msg.header.stamp = self.get_clock().now().to_msg()

    msg.info.resolution = self.resolution
    msg.info.width = self.width
    msg.info.height = self.height

    msg.info.origin = Pose()
    msg.info.origin.position.x = self.origin_x
    msg.info.origin.position.y = self.origin_y
    msg.info.origin.position.z = 0.0
    msg.info.origin.orientation.w = 1.0

    grid = [0 for _ in range(self.width * self.height)]

    # Outer walls
    self.set_obstacle_rect(grid, 0.0, -3.0, 6.0, 0.2)
    self.set_obstacle_rect(grid, 0.0, 3.0, 6.0, 0.2)
    self.set_obstacle_rect(grid, -3.0, 0.0, 0.2, 6.0)
    self.set_obstacle_rect(grid, 3.0, 0.0, 0.2, 6.0)

    # Inner obstacles
    self.set_obstacle_rect(grid, -1.0, 0.8, 2.0, 0.2)
    self.set_obstacle_rect(grid, 1.0, -0.8, 2.0, 0.2)

    msg.data = grid

    self.map_pub.publish(msg)
    self.get_logger().info('Published pre-known classroom map')
```

```
def main(args=None):
    rclpy.init(args=args)
    node = KnownMapPublisher()
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()
```

A* PATH PLANNING

Create A* Planner Node:

nano ~/ros2_ws/src/workshop_planner/workshop_planner/astar_map_planner.py

```
import math
import heapq
import rclpy

from rclpy.node import Node
from nav_msgs.msg import OccupancyGrid, Path, Odometry
from geometry_msgs.msg import PoseStamped, PoseWithCovarianceStamped
```

```
class AStarMapPlanner(Node):
```

```
    def __init__(self):
        super().__init__('astar_map_planner')
```

```
        # World coordinates in meters
        self.robot_x = None
        self.robot_y = None
        self.goal_world = None
```

```
        self.map_sub = self.create_subscription(
            OccupancyGrid,
            '/map',
            self.map_callback,
            10
        )
```

```
        self.path_pub = self.create_publisher(
            Path,
            '/planned_path',
            10
        )
```

```
        self.goal_sub = self.create_subscription(PoseStamped, '/goal_pose', self.goal_callback, 10)
```

```
        self.odom_sub = self.create_subscription(Odometry, '/odom', self.odom_callback, 10)
```

```
        self.map_received = False
        self.grid = []
        self.width = 0
        self.height = 0
        self.resolution = 0.0
        self.origin_x = 0.0
        self.origin_y = 0.0
```

```
        self.timer = self.create_timer(1.0, self.plan_loop)
```

```
    def odom_callback(self, msg):
        self.robot_x = msg.pose.pose.position.x
        self.robot_y = msg.pose.pose.position.y
```

```
    def goal_callback(self, msg):
        self.goal_world = (
            msg.pose.position.x,
            msg.pose.position.y
        )
        self.get_logger().info(f'Goal set: {self.goal_world}')
```

```
    def map_callback(self, msg):
        self.width = msg.info.width
        self.height = msg.info.height
        self.resolution = msg.info.resolution
        self.origin_x = msg.info.origin.position.x
        self.origin_y = msg.info.origin.position.y
```

```
        self.grid = list(msg.data)
        self.map_received = True
```

```
    def world_to_grid(self, x, y):
        gx = int((x - self.origin_x) / self.resolution)
        gy = int((y - self.origin_y) / self.resolution)
        return gx, gy
```

```
    def grid_to_world(self, gx, gy):
        x = self.origin_x + (gx + 0.5) * self.resolution
        y = self.origin_y + (gy + 0.5) * self.resolution
        return x, y
```

```
    def index(self, gx, gy):
        return gy * self.width + gx
```

```
    def is_free(self, gx, gy):
        if gx < 0 or gy < 0 or gx >= self.width or gy >= self.height:
            return False
```

```
        value = self.grid[self.index(gx, gy)]
```

```
        # 0 = free, 100 = obstacle, -1 = unknown
        return value == 0
```

```
    def heuristic(self, a, b):
        dy = abs(a[1] - b[1])
        return math.sqrt(2) * min(dx, dy) + abs(dx - dy)
```

```
    def get_neighbors(self, node):
        gx, gy = node
```

```
        directions = [
            (1, 0, 1.0),
            (-1, 0, 1.0),
            (0, 1, 1.0),
            (0, -1, 1.0),
            (1, 1, math.sqrt(2)),
            (1, -1, math.sqrt(2)),
            (-1, 1, math.sqrt(2)),
            (-1, -1, math.sqrt(2)),
        ]
```

```
        neighbors = []
```

```
        for dx, dy, cost in directions:
            nx = gx + dx
            ny = gy + dy
```

```
            if self.is_free(nx, ny):
                neighbors.append((nx, ny), cost)
```

```
        return neighbors
```

A* PATH PLANNING

Create A* Planner Node:

```
def astar(self, start, goal):
    open_heap = []
    heapq.heappush(open_heap, (0.0, start)) # auto sort for us

    came_from = {}
    g_score = {start: 0.0}
    closed_set = set()

    while open_heap:
        _, current = heapq.heappop(open_heap)

        if current in closed_set:
            continue

        if current == goal:
            return self.reconstruct_path(came_from, current)

        closed_set.add(current)

        for neighbor, move_cost in self.get_neighbors(current):
            tentative_g = g_score[current] + move_cost

            if neighbor not in g_score or tentative_g < g_score[neighbor]:
                came_from[neighbor] = current
                g_score[neighbor] = tentative_g

                f_score = tentative_g + self.heuristic(neighbor, goal)
                heapq.heappush(open_heap, (f_score, neighbor))

    return []

def reconstruct_path(self, came_from, current):
    path = [current]

    while current in came_from:
        current = came_from[current]
        path.append(current)

    path.reverse()
    return path
```

```
def publish_path(self, grid_path):
    path_msg = Path()
    path_msg.header.frame_id = 'odom'
    path_msg.header.stamp = self.get_clock().now().to_msg()

    for gx, gy in grid_path:
        x, y = self.grid_to_world(gx, gy)

        pose = PoseStamped()
        pose.header.frame_id = 'odom'
        pose.header.stamp = self.get_clock().now().to_msg()

        pose.pose.position.x = x
        pose.pose.position.y = y
        pose.pose.position.z = 0.05
        pose.pose.orientation.w = 1.0

        path_msg.poses.append(pose)

    self.path_pub.publish(path_msg)
```

```
def plan_loop(self):
    self.get_logger().warn('Waiting for /map...')
    return

    if self.robot_x is None or self.robot_y is None:
        self.get_logger().warn('Waiting for /odom...')
        return

    if self.goal_world is None:
        self.get_logger().warn('Waiting for RViz 2D Goal Pose...')
        return

    start = self.world_to_grid(self.robot_x, self.robot_y)

    goal = self.world_to_grid(
        self.goal_world[0],
        self.goal_world[1]
    )

    if not self.is_free(start[0], start[1]):
        self.get_logger().error(f'Start cell blocked: {start}')
        return

    if not self.is_free(goal[0], goal[1]):
        self.get_logger().error(f'Goal cell blocked: {goal}')
        return

    grid_path = self.astar(start, goal)

    if not grid_path:
        self.get_logger().error('No path found')
        return

    self.publish_path(grid_path)

    self.get_logger().info(
        f'Published path: {len(grid_path)} points | start={start} goal={goal}'
    )

def main(args=None):
    rclpy.init(args=args)
    node = AStarMapPlanner()
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()
```

A* PATH PLANNING

Create Path Following:

nano ~/ros2_ws/src/workshop_controller/workshop_controller/waypoint_follower.py

```
import math
import rclpy

from rclpy.node import Node
from nav_msgs.msg import Path, Odometry
from geometry_msgs.msg import Twist
from tf.transformations import euler_from_quaternion
```

```
class WaypointFollower(Node):
```

```
    def __init__(self):
        super().__init__('waypoint_follower')
```

```
        self.path = []
        self.current_waypoint = 0
```

```
        self.x = 0.0
        self.y = 0.0
        self.yaw = 0.0
```

```
        self.path_sub = self.create_subscription(
            Path,
            '/planned_path',
            self.path_callback,
            10
        )
```

```
        self.odom_sub = self.create_subscription(
            Odometry,
            '/odom',
            self.odom_callback,
            10
        )
```

```
        self.cmd_pub = self.create_publisher(
            Twist,
            '/cmd_vel',
            10
        )
```

```
        self.timer = self.create_timer(0.1, self.control_loop)
```

```
    def path_callback(self, msg):
        self.path = []
```

```
        for pose in msg.poses:
            self.path.append((
                pose.pose.position.x,
                pose.pose.position.y
            ))
```

```
        self.current_waypoint = 0
        self.get_logger().info(f'Received path with {len(self.path)} waypoints')
```

```
    def odom_callback(self, msg):
        self.x = msg.pose.pose.position.x
        self.y = msg.pose.pose.position.y
```

```
        q = msg.pose.pose.orientation
        quat = [q.x, q.y, q.z, q.w]

        _, _, self.yaw = euler_from_quaternion(quat)
```

```
    def normalize_angle(self, angle):
        while angle > math.pi:
            angle -= 2.0 * math.pi
        while angle < -math.pi:
            angle += 2.0 * math.pi
        return angle
```

```
    def control_loop(self):
        if len(self.path) == 0:
            return
```

```
        if self.current_waypoint >= len(self.path):
            self.stop_robot()
            return
```

```
        target_x, target_y = self.path[self.current_waypoint]
```

```
        dx = target_x - self.x
        dy = target_y - self.y
```

```
        distance = math.sqrt(dx * dx + dy * dy)
```

```
        if distance < 0.15:
            self.current_waypoint += 1
            return
```

```
        target_yaw = math.atan2(dy, dx)
        yaw_error = self.normalize_angle(target_yaw - self.yaw)
```

```
        cmd = Twist()
```

```
        if abs(yaw_error) > 0.5:
            cmd.linear.x = 0.03
        else:
            cmd.linear.x = 0.15
```

```
        cmd.angular.z = 1.5 * yaw_error
```

```
        max_angular = 1.2
        cmd.angular.z = max(-max_angular, min(max_angular, cmd.angular.z))
```

```
        self.cmd_pub.publish(cmd)
```

```
    def stop_robot(self):
        cmd = Twist()
        self.cmd_pub.publish(cmd)
```

```
def main(args=None):
    rclpy.init(args=args)
    node = WaypointFollower()
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()
```

A* PATH PLANNING

Run all Nodes:

```
# T1: Gazebo
ros2 launch workshop_gazebo classroom_world.launch.py

# T2: Map publisher
ros2 run workshop_mapping known_map_publisher

# T3: A* planner
ros2 run workshop_planner astar_map_planner

# T4: Waypoint follower
ros2 run workshop_controller waypoint_follower

# T5: RViz2 (to visualise the path)
rviz2
```

A* PATH PLANNING

Make package for launch files and launch files:

```
cd ~/ros2_ws/src
```

```
ros2 pkg create workshop_bringup --build-type ament_python
```

```
cd workshop_bringup
```

```
mkdir launch
```

```
nano /launch/full_system.launch.py
```

```
import os

from launch import LaunchDescription
from launch.actions import IncludeLaunchDescription, TimerAction
from launch.launch_description_sources import PythonLaunchDescriptionSource
from launch_ros.actions import Node

from ament_index_python.packages import get_package_share_directory

def generate_launch_description():
    gazebo_launch = os.path.join(
        get_package_share_directory('workshop_gazebo'),
        'launch',
        'classroom_world.launch.py'
    )
```

```
    gazebo = IncludeLaunchDescription(
        PythonLaunchDescriptionSource(gazebo_launch)
    )
```

```
    known_map = Node(
        package='workshop_mapping',
        executable='known_map_publisher',
        name='known_map_publisher',
        output='screen'
    )
```

```
    astar = Node(
        package='workshop_planner',
        executable='astar_map_planner',
        name='astar_map_planner',
        output='screen'
    )
```

```
    waypoint_follower = Node(
        package='workshop_controller',
        executable='waypoint_follower',
        name='waypoint_follower',
        output='screen'
    )
```

```
    return LaunchDescription([
        gazebo,
        TimerAction(period=3.0, actions=[known_map]),
        TimerAction(period=4.0, actions=[astar]),
        TimerAction(period=5.0, actions=[waypoint_follower]),
        TimerAction(period=6.0, actions=[rviz]),
    ])
```

A* PATH PLANNING

Register launch files to setup.py

nano ~/ros2_ws/src/workshop_bringup /workshop_bringup/setup.py

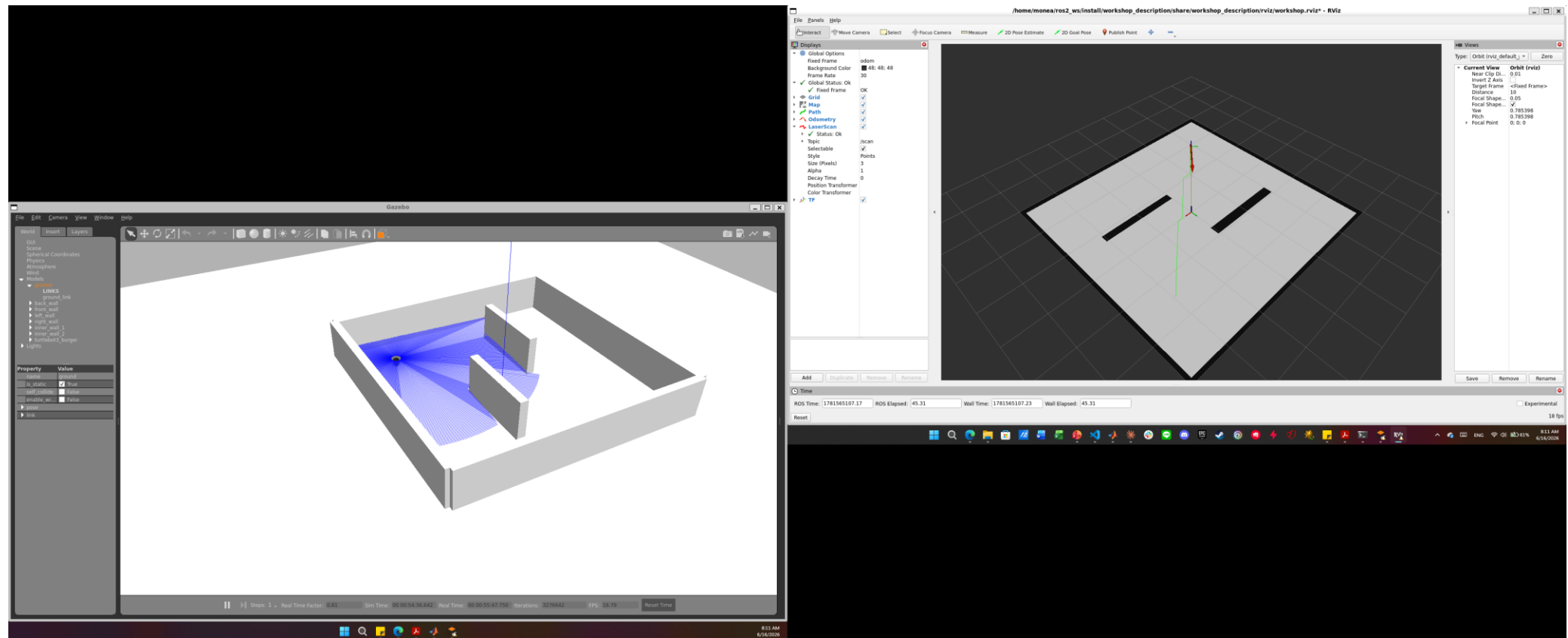
```
from setuptools import find_packages, setup
import os
from glob import glob

package_name = 'workshop_bringup'

setup(
    name=package_name,
    version='0.0.0',
    packages=find_packages(exclude=['test']),
    data_files=[
        ('share/ament_index/resource_index/packages',
         ['resource/' + package_name]),
        ('share/' + package_name, ['package.xml']),
        (os.path.join('share', package_name, 'launch'), glob('launch/*.launch.py')),
    ],
    install_requires=['setuptools'],
    zip_safe=True,
    maintainer='admins',
    maintainer_email='admins@todo.todo',
    description='TODO: Package description',
    license='TODO: License declaration',
    extras_require={
        'test': [
            'pytest',
        ],
    },
    entry_points={
        'console_scripts': [
        ],
    },
)
```

A* PATH PLANNING

Launch all files:



REPORT:

Use a different robot (TurtleBot3 Waffle Pi with a camera) and must complete one lap around a custom race-track using only camera input.

